

Getting Started with Vue.js

Over the past few years, few web frameworks have survived. Vue.js has not only survived but thrived. Let's see why.

Kshitij Sobti | feedback@digit.in

The world of web development is one of constant churn; new frameworks pop up one day and take the world by storm, only to disappear in a month. If you are looking to create an actual app that is intended to be deployed for people to use, it is important to consider the history of a project and how active its development is at the moment.

Vue.js has been around for over three years, and in that time it has gained a lot in popularity for its simplicity, minimalism and performance. It calls itself 'The Progressive JavaScript Framework' and markets itself with the keywords Approachable, Versatile and Performant.

As you may have derived from the name, Vue.js mostly concerns itself with the View part of your application. It lets you create reusable components that can be composed to build powerful, reactive user interfaces. Like many modern web frameworks, Vue.js uses a virtual DOM to make the framework fast and responsive.

It is not concerned with how you connect to your API, how you fetch

data. It doesn't enforce any structure to your code or concern itself with your business logic. It doesn't even enforce a structure for your Vue.js code.

In fact you can create a working Vue.js app in a single JavaScript file that is loaded directly in your HTML page, along with the Vue.js code with no requirement for a build step. Of course, if you want to use the latest JavaScript syntax and want to bundle your code better you will still need something like Webpack.

We will use some modern JavaScript features that require a somewhat modern browser. Any recent version on Chrome, Firefox, Safari or Edge will work – Internet Explorer will not.

We will be building a simple app that lets you search for users on GitHub using their public API.

SETTING UP OUR CODE

We will set up our Vue app the simple way, without using any external tools. We will simply create an HTML file that includes Vue and our code, and optionally some CSS to make the application presentable.

Create a fresh directory to play around in, and in it create an empty HTML file called index.html and a JavaScript file called app.js. Of course you can call these files whatever you want, but you will need to adjust the code examples accordingly.

You can paste in any simple boilerplate HTML5 code. All you need in the header section is this script tag that will load Vue from a CDN:

```
<script src="https://unpkg.com/vue"></script>
```

And inside the body tag create a single div tag where the app will be mounted, and a script tag to include our app code:

```
<div id="app"> </div>  
<script src="app.js"></script>
```

If you open this HTML file in the browser and open the console, you should see output like "You are running Vue in development mode. ..."

VUE.JS BASICS

Before we delve into the code, there are a few things you should know about how Vue works. Since Vue is a framework for building user interfaces in a web browser, at some point it needs to output HTML that will be rendered to the page.

Different frameworks have different approaches for how this output is generated, and the approach Vue uses is that of templates. You provide Vue some template HTML code that has special markup for where to fill-in data, and Vue will not only fill in the data you provide but also create a binding so any changes to your data will automatically update the relevant parts of the UI.

Let's look at the simplest example of a "Hello World" app here: <http://digit.in/HlwWrlD>.

As you can see in the code example above, we are creating a new Vue instance and providing it a template for our output via **template**, the data to fill into the template via **data** and the HTML element in our page – where this Vue component is to be rendered – via **el**. Setting **el** to '#app' tells it to mount the component in the element with an id of **app**; Vue will then render this component and place it where the we put the div with an id of **app**.

The template above is an HTML code with a placeholder for message defined by putting it inside double curly-braces. Vue will look at the value of **greeting** in data and fill it in the placeholder inside the **h1** tag.